

university of groningen

Performance Engineering: Lecture #03

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-07

Overview



1. Case Study: `inc2`
2. Case Study: `vreduce`
3. Other Compiler Optimizations
4. Compiler Reports
5. Final

Content



1. **Case Study:** `inc2`
2. Case Study: `vreduce`
3. Other Compiler Optimizations
4. Compiler Reports
5. Final

Case Study: `inc2`



- Compilers must be careful to apply only safe optimizations to a code
- Purpose of this case study:
  - Study consequence of compiler decisions based on lacking knowledge
  - Understand the **memory aliasing** problem
  - Explore a remedy
- Specification for `inc2`:
  - Function does not return anything, input and output are provided as C pointers
  - The function adds twice the input integer to the output integer

inc2: Implementations (1/3)



- Two implementations:

```
1  void inc2a(int *xp, int *yp)
2  {
3      *xp += *yp;
4      *xp += *yp;
5  }
6
7  void inc2b(int *xp, int *yp)
8  {
9      *xp += 2 * (*yp);
10 }
```

- **Question**: Will both codes return the same answer?

inc2: Implementations (2/3)



- Two implementations:

```
1  void inc2a(int *xp, int *yp)
2  {
3      *xp += *yp;
4      *xp += *yp;
5  }
6
7  void inc2b(int *xp, int *yp)
8  {
9      *xp += 2 * (*yp);
10 }
```

- **Answer**: NO
- Value stored in `*xp` depends on whether the pointers are the same or different

	different	same
`inc2a`	$x \leftarrow x + 2y$	$x \leftarrow 4x$
`inc2b`	$x \leftarrow x + 2y$	$x \leftarrow 3x$

inc2: Implementations (3/3)



Code for inc2a():

```
1  inc2a:
2  .LFB0:
3          .cfi_startproc
4          endbr64
5          movl    (%rsi), %eax
6          addl    (%rdi), %eax
7          movl    %eax, (%rdi)
8          addl    (%rsi), %eax
9          movl    %eax, (%rdi)
10         ret
11         .cfi_endproc
```

Code for inc2b():

```
1  inc2b:
2  .LFB1:
3          .cfi_startproc
4          endbr64
5          movl    (%rsi), %eax
6          addl    %eax, %eax
7          addl    %eax, (%rdi)
8          ret
9          .cfi_endproc
```

Memory Aliasing



- **Situation**: The same memory location can be accessed through more than one symbolic name (like a variable or a pointer) in a program
- **Problem**: Various optimizations are impossible if memory aliasing cannot be excluded
  - See the `inc2` example
- **Solution**: Avoid memory aliasing by providing sufficient information to the compiler
  - Use the `restrict` keyword in C

inc2: Optimization



- The memory aliasing issue can be solved by using `restrict`-qualified pointers
- By using this qualifier, one promises to the compiler that the object, to which the pointer points, will only be accessed through this pointer

Modified C-Code:

```
1 void inc2c(int *restrict xp,
2             int *restrict yp)
3 {
4     *xp += *yp;
5     *xp += *yp;
6 }
```

Assembler:

```
1 inc2o:
2 .LFB0:
3         .cfi_startproc
4         endbr64
5         movl    (%rsi), %eax
6         addl    %eax, %eax
7         addl    %eax, (%rdi)
8         ret
9         .cfi_endproc
```

Content



1. Case Study: `inc2`
2. **Case Study: `vreduce`**
3. Other Compiler Optimizations
4. Compiler Reports
5. Final

Case Study: `vreduce`



- **Purpose** of this case study: Explore different loop-related optimization strategies
- **Specification** for `vreduce`:
  - Use a data type for arrays of numbers with a length fixed at run-time
  - Provide a function that computes the sum of all array elements using a pointer to an array on input and a pointer referencing the output
  - Different simple data types should be supported

vreduce: Vector Data Type (1/3)



- The following information is stored in a file `myvector.h`
- Data structure:

```c
#include <stdlib.h>

#define DTYPE double

typedef struct {
    long int  len;
    DTYPE     *data;
} vec_rec, *vec_ptr;
```

`vreduce`: Vector Data Type (2/3)



- Data structure **creator**:

```
11 vec_ptr new_vec(long int len)
12 {
13     /* Allocate header structure */
14     vec_ptr result = (vec_ptr) malloc(sizeof(vec_rec));
15     if (!result)
16         return NULL;                  /* Allocation failed */
17     result->len = len;
18
19     /* Allocate array */
20     if (len > 0) {
21         DTYPE *data = (DTYPE *) calloc(len, sizeof(DTYPE));
22         if (!data) {
23             free((void *) result);
24             return NULL;              /* Allocation failed */
25         }
26         result->data = data;
27     }
28     else
29         result->data = NULL;
30
31     return result;
32 }
```

`vreduce`: Vector Data Type (3/3)



- Array **element accessor**:

```
35 int get_vec_element(vec_ptr v, long int index, DTYPE *dest)
36 {
37     if (index < 0 || index >= v->len)
38         return 0;
39     *dest = v->data[index];
40     return 1;
41 }
```

- Array **length accessor**:

```
44 long int get_vec_length(vec_ptr v)
45 {
46     return v->len;
47 }
```

- Array **data accessor**:

```
50 DTYPE* get_vec_data(vec_ptr v)
51 {
52     return v->data;
53 }
```

`vreduce`: 1st Implementation (1/2)



- Our first implementation:

```
1 #include "myvector.h"
2
3 void vreduce1(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6
7      *dest = 0;
8      for (i = 0; i < get_vec_length(v); i++) {
9          DTYPE val;
10         get_vec_element(v, i, &val);
11         *dest += val;
12     }
13 }
```

- **Performance challenge**: **function calls**

`vreduce`: 1st Implementation (2/2)



- Compilation using -O0:

```
1  #include "myvector.h"
2
3  void vreduce1(vec_ptr v, DTYPE *dest)
4   {
5       long int i;
6
7       *dest = 0;
8       for (i = 0; i < get_vec_length(v); i++) {
9           DTYPE val;
10          get_vec_element(v, i, &val);
11          *dest += val;
12      }
13  }
```

- **Observation(s)**:
  - Two function calls
  - Number of instructions: 19

```
159         jmp     .L16
160 .L17:
161         leaq    -24(%rbp), %rdx
162         movq    -16(%rbp), %rcx
163         movq    -40(%rbp), %rax
164         movq    %rcx, %rsi
165         movq    %rax, %rdi
166         call    get_vec_element
167         movq    -48(%rbp), %rax
168         movsd   (%rax), %xmm1
169         movsd   -24(%rbp), %xmm0
170         addsd   %xmm1, %xmm0
171         movq    -48(%rbp), %rax
172         movsd   %xmm0, (%rax)
173         addq    $1, -16(%rbp)
174 .L16:
175         movq    -40(%rbp), %rax
176         movq    %rax, %rdi
177         call    get_vec_length
178         cmpq    %rax, -16(%rbp)
179         jl      .L17
```

`vreduce`: 2nd Implementation (1/2)



- **First remedy**: Store array length in local variable:

```
1  #include "myvector.h"
2
3  void vreduce2(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6      long int n = get_vec_length(v);
7
8      *dest = 0;
9      for (i = 0; i < n; i++) {
10         DTYPE val;
11         get_vec_element(v, i, &val);
12         *dest += val;
13     }
14 }
```

`vreduce`: 2nd Implementation (2/2)



- Compilation using `-O0`:
- **Observation(s)**:
  - Only one function call
  - Number of instructions: 17

```
163         jmp     .L16
164 .L17:
165         leaq    -32(%rbp), %rdx
166         movq    -24(%rbp), %rcx
167         movq    -40(%rbp), %rax
168         movq    %rcx, %rsi
169         movq    %rax, %rdi
170         call    get_vec_element
171         movq    -48(%rbp), %rax
172         movsd   (%rax), %xmm1
173         movsd   -32(%rbp), %xmm0
174         addsd   %xmm1, %xmm0
175         movq    -48(%rbp), %rax
176         movsd   %xmm0, (%rax)
177         addq    $1, -24(%rbp)
178 .L16:
179         movq    -24(%rbp), %rax
180         cmpq    -16(%rbp), %rax
181         jl      .L17
```

`vreduce`: 3rd Implementation (1/2)



- **Second remedy**: Access data directly:

```
1  #include "myvector.h"
2
3  void vreduce3(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6      long int length = get_vec_length(v);
7      DTYPE    *data  = get_vec_data(v);
8
9      *dest = 0;
10     for (i = 0; i < length; i++) {
11         *dest += data[i];
12     }
13 }
```

`vreduce`: 3rd Implementation (2/2)



- Compilation using `-O0`:
- **Observation(s)**:
  - No function calls left
  - Number of instructions: 15

```
164             jmp     .L16
165     .L17:
166             movq    -48(%rbp), %rax
167             movsd   (%rax), %xmm1
168             movq    -24(%rbp), %rax
169             leaq    0(,%rax,8), %rdx
170             movq    -8(%rbp), %rax
171             addq    %rdx, %rax
172             movsd   (%rax), %xmm0
173             addsd   %xmm1, %xmm0
174             movq    -48(%rbp), %rax
175             movsd   %xmm0, (%rax)
176             addq    $1, -24(%rbp)
177     .L16:
178             movq    -24(%rbp), %rax
179             cmpq    -16(%rbp), %rax
180             jl      .L17
```

`vreduce`: 4th Implementation (1/2)



- **Next optimization**: Use local variable for accumulation:

```
1  #include "myvector.h"
2
3  void vreduce4(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6      long int length = get_vec_length(v);
7      DTYPE    *data  = get_vec_data(v);
8      DTYPE    accum  = 0;
9
10     for (i = 0; i < length; i++) {
11         accum += data[i];
12     }
13     *dest = accum;
14 }
```

`vreduce`: 4th Implementation (2/2)



- Compilation using `-O0`:
- **Observation(s)**:
  - No function calls left
  - Number of instructions: 13

```
163         jmp     .L16
164 .L17:
165         movq    -32(%rbp), %rax
166         leaq    0(,%rax,8), %rdx
167         movq    -8(%rbp), %rax
168         addq    %rdx, %rax
169         movsd   (%rax), %xmm0
170         movsd   -24(%rbp), %xmm1
171         addsd   %xmm1, %xmm0
172         movsd   %xmm0, -24(%rbp)
173         addq    $1, -32(%rbp)
174 .L16:
175         movq    -32(%rbp), %rax
176         cmpq    -16(%rbp), %rax
177         jl      .L17
```

vreduce: Back to 1st Implementation



- Compilation using -O3:

```
1  #include "myvector.h"
2
3  void vreduce1(vec_ptr v, DTYPE *dest)
4   {
5       long int i;
6
7       *dest = 0;
8       for (i = 0; i < get_vec_length(v); i++) {
9           DTYPE val;
10          get_vec_element(v, i, &val);
11          *dest += val;
12      }
13  }
```

- **Observation(s)**:
  - No function calls left
  - Further reduction of the number of instructions

```
115 vreduce1:
116 .LFB20:
117         .cfi_startproc
118         endbr64
119         movq    (%rdi), %rdx
120         movq    $0x000000000, (%rsi)
121         testq   %rdx, %rdx
122         jle     .L20
123         movq    8(%rdi), %rax
124         pxor    %xmm0, %xmm0
125         leaq    (%rax,%rdx,8), %rdx
126         .p2align 4,,10
127         .p2align 3
128 .L22:
129         addsd   (%rax), %xmm0
130         addq    $8, %rax
131         movsd   %xmm0, (%rsi)
132         cmpq    %rax, %rdx
133         jne     .L22
134 .L20:
135         ret
136         .cfi_endproc
```

Function Inlining



- **Transformation**: A function call is replaced by the body of the function itself
- **Prerequisite**: Function body code must be available
- **Hints**:
  - The `inline` qualifier is typically not required
  - The `inline` qualifier does not enforce inlining
  - Inlining can be enforced using compiler-specific function declaration attributes:
    - GCC: `__attribute__((always_inline))`

Content



1. Case Study: `inc2`
2. Case Study: `vreduce`
3. **Other Compiler Optimizations**
4. Compiler Reports
5. Final

Constant Folding and Constant Propagation



- **Constant folding**:
  - **Transformation**: Replace an arithmetic expression at compile time, e.g. `i = 5 * 12` → `i = 60`
  - Caveat: Evaluation of arithmetic expressions on systems used for compilation and execution may differ
- **Constant propagation**:
  - **Transformation**: Substitute the values of known constants in expressions at compile time

```
1 int f() {
2     int i = 3 * 15;
3     int j = 3 * 7 - 42 / 2;
4     return (i * j);
5 }
```

→

```
1 int f() {
2     return 0;
3 }
```

Common Subexpression Elimination



- **Transformation**:
  - Identify instances of identical expressions
  - Evaluate expression only once and store result in a temporary variable
  - Reuse this temporary variable
- Example:

```c
int f(int a, int b, int c) {
    int x, y;

    x = a * b + c;
    y = a * b + 2 * c;

    /* more code */

    return (x + y);
}
```

→

```c
int f(int a, int b, int c) {
    int x, y, t;

    t = a * b;
    x = t + c;
    y = t + 2 * c;

    /* more code */

    return (x + y);
}
```

Dead Code Elimination



- **Transformation**: Remove code using dead variables
- **Dead variables**: A variable is said to be dead if it is never used after its last definition
- **Example** compiled using `gcc-14 -O3`:

```
1  void f() {
2      int i;
3      for (i = 0; i < 128; i++);
4  }
5
6  int main() {
7      f();
8      return 0;
9  }
```

```
6  f:
7  .LFB0:
8          .cfi_startproc
9          endbr64
10         ret
11         .cfi_endproc
12 .LFE0:
13         .size   f, .-f
14         .section        .text.startup,"ax",@progbits
15         .p2align 4
16         .globl  main
17         .type   main, @function
18 main:
19 .LFB1:
20         .cfi_startproc
21         endbr64
22         xorl    %eax, %eax
23         ret
24         .cfi_endproc
```

Optimizations Impacting Floating-Point Operations



- Compilers support optimizations on instructions for floating-point operations that affect results
  - Results may not any more comply to IEEE standards
  - Typically, this concerns special cases like subnormal numbers or floating-point values `NaN`, `Inf`
- Example GCC: `-ffast-math`
  - Includes `-fno-math-errno` that suppresses setting of `errno`, which may allow avoiding function calls

☞ **Trade precision for speed**

Machine Specific Optimizations



- Compilers may use apply different optimizations or may apply optimizations differently when the target architecture is known
  - Example: Use of specific instructions
- Relevant GCC options:
  - `-mtune=<cpu-type>`: Tune code for given CPU architecture while keeping it portable
  - `-march=<cpu-type>`: Allow compiler to generate code that may not run on processors apart from the one indicated
    - Code generated using `-march=skylake-avx512` might only run on Intel Skylake processors
    - Code generated using `-march=armv8-a+sve` might only run on Arm-based processors supporting both Arm-v8 and SVE
  - Set `cpu-type` to `native` to let the compiler autodetect the current platform

Content



1. Case Study: `inc2`
2. Case Study: `vreduce`
3. Other Compiler Optimizations
4. **Compiler Reports**
5. Final

Compiler Reports: Introduction



- Compilers can generate optimization reports
- **Caveat**: Some of the reports are more suitable for compiler developers
- Inspecting compiler optimization reports is often easier than analysis of the generated assembler

GCC Optimization Reports



- Relevant GCC options:
  - `-fopt-info`: Write default optimization information
  - `-fopt-info-<selector>`: Write selected information
  - `-fopt-info-<selector>=<file>`: Write selected information to given file
    - Selector = `missed`, `loop`, `inline`, `vec`, `missed`, `optall`, ...

GCC Optimization Reports: Example (1/2)



Example code:

```
11 vec_ptr new_vec(long int len)
12 {
13     /* Allocate header structure */
14     vec_ptr result = (vec_ptr) malloc(sizeof(vec_rec));
15     if (!result)
16         return NULL;
17     result->len = len;
18
19     /* Allocate array */
20     if (len > 0) {
21         DTYPE *data = (DTYPE*) calloc(len,sizeof(DTYPE));
22         if (!data) {
23             free((void *) result);
24             return NULL;
25         }
26         result->data = data;
27     }
28     else
29         result->data = NULL;
30
31     return result;
32 }
```

```
1  #include "myvector.h"
2
3  void vreduce1(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6
7      *dest = 0;
8      for (i = 0; i < get_vec_length(v); i++) {
9          DTYPE val;
10         get_vec_element(v, i, &val);
11         *dest += val;
12     }
13 }
```

GCC Optimization Reports: Example (2/2)



Optimization report: `gcc-14 -c -O1 -fopt-info-missed vreduce1.c`

```
1 vreduce1.c:8:22: missed:   not inlinable: vreduce1/20 -> get_vec_length/18, function not inline candidate
2 vreduce1.c:10:10: missed:   not inlinable: vreduce1/20 -> get_vec_element/17, function not inline candidate
3 myvector.h:23:13: missed:   not inlinable: new_vec/16 -> free/23, function body not available
4 myvector.h:21:33: missed:   not inlinable: new_vec/16 -> calloc/22, function body not available
5 myvector.h:14:32: missed:   not inlinable: new_vec/16 -> malloc/21, function body not available
```

LLVM clang Optimization Reports



- Relevant clang options:
  - `-Rpass=<arg>`: Report transformations performed
  - `-Rpass-missed=<arg>`: Report missed transformations
  - `-Rpass-analysis=<arg>`: Report transformation analysis
    - arg = POSIX regular expression

clang Optimization Reports: Example (1/2)



Example code:

```
11 vec_ptr new_vec(long int len)
12 {
13     /* Allocate header structure */
14     vec_ptr result = (vec_ptr) malloc(sizeof(vec_rec));
15     if (!result)
16         return NULL;
17     result->len = len;
18
19     /* Allocate array */
20     if (len > 0) {
21         DTYPE *data = (DTYPE*) calloc(len,sizeof(DTYPE));
22         if (!data) {
23             free((void *) result);
24             return NULL;
25         }
26         result->data = data;
27     }
28     else
29         result->data = NULL;
30
31     return result;
32 }
```

```
1  #include "myvector.h"
2
3  void vreduce1(vec_ptr v, DTYPE *dest)
4  {
5      long int i;
6
7      *dest = 0;
8      for (i = 0; i < get_vec_length(v); i++) {
9          DTYPE val;
10         get_vec_element(v, i, &val);
11         *dest += val;
12     }
13 }
```

clang Optimization Reports: Example (2/2)

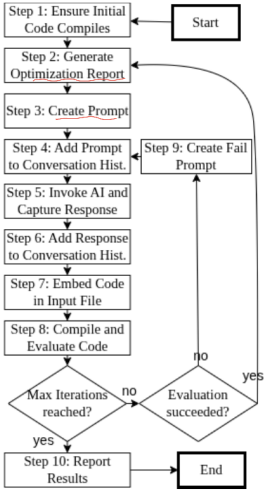


Optimization report: `clang-19 -c -O1 -Rpass-missed=.* vreduce1.c`

```
vreduce1.c:11:16: remark: failed to move load with loop-invariant address because the loop may invalidate its value [-Rpass-missed=licm]
   11 |           *dest += val;
      |                ^
vreduce1.c:11:16: remark: failed to hoist load with loop-invariant address because load is conditionally executed [-Rpass-missed=licm]
vreduce1.c:11:16: remark: failed to move load with loop-invariant address because the loop may invalidate its value [-Rpass-missed=licm]
vreduce1.c:8:6: remark: loop not vectorized [-Rpass-missed=loop-vectorize]
    8 |       for (i = 0; i < get_vec_length(v); i++) {
      |       ^
In file included from vreduce1.c:1:
./myvector.h:11:1: remark: 7 virtual registers copies 2.708984e+00 total copies cost generated in function [-Rpass-missed=regalloc]
   11 | vec_ptr new_vec(long int len)
      | ^
```

Compiler Reports Processing Using LLMs

- Recent research result: **CompilerGPT** [Pirkelbauer, Liao; 2025]
- **Idea**: Use compiler oztimisation reports as input for **optimized code generation by an LLM**



Content



1. Case Study: `inc2`
2. Case Study: `vreduce`
3. Other Compiler Optimizations
4. Compiler Reports
5. **Final**

Summary Lecture #03



- Manual optimization techniques
  - Avoid memory aliasing problem
  - Eliminate function calls in loops
- Compiler optimization techniques
  - Constant folding and propagation
  - Common subexpression elimination
  - Dead code elimination
  - Floating-point operation optimizations
  - Machine-specific optimizations
- Compiler optimization reports

Questions?



[Wiki Media]